EXPERIMENT 4: ARTIFICIAL NEURAL NETWORK WITH BACKPROPAGATION

Experiment Question: Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Backpropagation: General concept and mathematical rule

An Artificial Neural Network (ANN) is made of layers of neurons. Each neuron computes a weighted sum of its inputs plus a bias and passes it through an activation function; here the sigmoid $\sigma(z) = 1/(1 + e^{-z})$ is used. The network in this experiment has one input layer, one hidden layer, and one output layer.

Backpropagation trains the network in two phases. In the forward pass the inputs are propagated through the hidden and output layers to produce the outputs. In the backward pass the error between the target and the output is propagated back through the network, and every weight is changed a little in the direction that reduces the error (gradient descent).

Output error term: $\delta_k = (t_k - a_k)\, a_k\, (1 - a_k)$. Hidden error term: $\delta_j = a_j\, (1 - a_j)\, \Sigma_k\, \delta_k\, w_{jk}$. Weight update: $w_{ij}(\text{new}) = w_{ij} + \eta\, \delta_j\, x_i$, where η is the learning rate.

In these programs the weights are updated after every training sample (online learning) and the training samples are shuffled in every epoch. The mean squared error (MSE) is printed at regular epochs to show learning.

Important: the weights start from random numbers (fixed here with seed 1), so the exact numbers in the output can differ if the seed, the learning rate, or the number of epochs is changed. Subproblems 4 and 5 load their datasets from scikit-learn (pip install scikit-learn).

Subproblem 1: XOR Gate Learning

1. Aim

To build a 2-4-1 neural network trained with the Backpropagation algorithm that learns the XOR function, which a single-layer perceptron cannot learn.

2. Concept

XOR is not linearly separable, so a hidden layer is required. The network has 2 inputs, 4 hidden neurons, and 1 output neuron with sigmoid activations.

3. Dataset Description

The training examples are the four input combinations of the XOR gate.

A	B	Target (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

4. Algorithm

1. Initialise the weights randomly between −1 and 1, set all biases to 0, and choose the learning rate and number of epochs.
2. Present one training sample and perform the forward pass: compute the hidden-layer outputs and then the output-layer outputs using the sigmoid function.
3. Compute the error at each output neuron and convert it to the output error term δ.
4. Propagate δ backwards through the weights to obtain the hidden-layer error terms.
5. Update every weight and bias by gradient descent: new value = old value + η × δ × input.
6. Repeat steps 2–5 for every training sample; one pass over the data is one epoch. Continue until the chosen number of epochs is complete.
7. Test the trained network on the test inputs and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Network: 2 input neurons, 4 hidden neurons, 1 output neuron(s); sigmoid activation; learning rate η = 0.5; weights start random in [−1, 1] (seed 1) and biases start at 0.
2. Take training sample 4: x = (1.0000, 1.0000), target t = (0.0000).
3. Hidden neuron h1: z = (−0.1660)(1.0000) + (−0.7065)(1.0000) + 0.0000 = −0.8724; a = σ(z) = 0.2947.
4. Hidden neuron h2: z = (0.4406)(1.0000) + (−0.8153)(1.0000) + 0.0000 = −0.3747; a = σ(z) = 0.4074.
5. Hidden neuron h3: z = (−0.9998)(1.0000) + (−0.6275)(1.0000) + 0.0000 = −1.6273; a = σ(z) = 0.1642.

1. The remaining 1 hidden neuron(s) are computed in the same way; the hidden vector is a = (0.2947, 0.4074, 0.1642, 0.3309).
2. Output neuron o1: z = Σ(a_h × w) + b = 0.0668; a = σ(z) = 0.5167.
3. Output error term δo1 = (t − a)·a·(1 − a) = (0.0000 − 0.5167)×0.5167×(1 − 0.5167) = −0.1290.
4. Hidden error term δh1 = a(1 − a)·Σ(δo × w) = 0.2947×(1 − 0.2947)×(−0.1290×−0.2065) = 0.0055.
5. Hidden error term δh2 = a(1 − a)·Σ(δo × w) = 0.4074×(1 − 0.4074)×(−0.1290×0.0776) = −0.0024.
6. Hidden error term δh3 = a(1 − a)·Σ(δo × w) = 0.1642×(1 − 0.1642)×(−0.1290×−0.1616) = 0.0029.
7. Weight update (output layer): w(h1→o1) new = −0.2065 + 0.5×−0.1290×0.2947 = −0.2255.
8. Weight update (hidden layer): w(x1→h1) new = −0.1660 + 0.5×0.0055×1.0000 = −0.1632. Biases are updated as b new = b + η×δ.
9. These forward and backward passes are repeated for every training sample in every epoch. The mean squared error falls from 0.019769 (epoch 1000) to 0.000885 (epoch 5000).
10. After training, the test inputs are passed forward only (no weight changes). Output > 0.5 (or the largest output neuron) gives the predicted class: accuracy = 100.00%.

6. Formulae

Notation: x_i = input, a_j = neuron output, t_k = target, η = learning rate, w_{ij} = weight from neuron i to neuron j.

- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$, with derivative $\sigma'(z) = a(1 - a)$
- Forward pass: $z_j = \Sigma\, w_{ij}\, x_i + b_j$; $a_j = \sigma(z_j)$
- Output error term: $\delta_k = (t_k - a_k)\, a_k\, (1 - a_k)$
- Hidden error term: $\delta_j = a_j\, (1 - a_j)\, \Sigma_k\, \delta_k\, w_{jk}$
- Weight update: $w_{ij}(\text{new}) = w_{ij} + \eta\, \delta_j\, x_i$; $b_j(\text{new}) = b_j + \eta\, \delta_j$
- Mean squared error: $MSE = (1/N)\, \Sigma\, (t - a)^2$

7. Python Program

```
import numpy as np

np.random.seed(1)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

class NeuralNetwork:
    def __init__(self, n_in, n_hidden, n_out, lr):
        self.W1 = np.random.uniform(-1, 1, (n_in, n_hidden))
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = np.random.uniform(-1, 1, (n_hidden, n_out))
        self.b2 = np.zeros((1, n_out))
```

```
        self.lr = lr

    def forward(self, X):
        self.a1 = sigmoid(X @ self.W1 + self.b1)       # hidden layer
        self.a2 = sigmoid(self.a1 @ self.W2 + self.b2)  # output layer
        return self.a2

    def backward(self, x, y):
        delta2 = (y - self.a2) * self.a2 * (1 - self.a2)          # output
error term
        delta1 = (delta2 @ self.W2.T) * self.a1 * (1 - self.a1)   # hidden
error term
        self.W2 += self.lr * self.a1.T @ delta2
        self.b2 += self.lr * delta2
        self.W1 += self.lr * x.T @ delta1
        self.b1 += self.lr * delta1

    def train(self, X, y, epochs, log_every):
        for epoch in range(1, epochs + 1):
            for i in np.random.permutation(len(X)):
                xi, yi = X[i:i + 1], y[i:i + 1]
                self.forward(xi)
                self.backward(xi, yi)
            if epoch % log_every == 0:
                mse = np.mean((y - self.forward(X)) ** 2)
                print("Epoch %5d   MSE = %.6f" % (epoch, mse))

# XOR gate: inputs and target outputs
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)

nn = NeuralNetwork(n_in=2, n_hidden=4, n_out=1, lr=0.5)
nn.train(X, y, epochs=5000, log_every=1000)

out = nn.forward(X)
print("\nInput    Target   Output   Predicted")
for xi, t, o in zip(X, y, out):
    print(xi.astype(int), "   ", int(t[0]), "    %.4f      %d" % (o[0], o[0]
> 0.5))
print("Accuracy = %.2f%%" % (np.mean((out > 0.5) == y) * 100))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/4.1.py ====
Epoch  1000   MSE = 0.019769
Epoch  2000   MSE = 0.003662
Epoch  3000   MSE = 0.001844
Epoch  4000   MSE = 0.001204
Epoch  5000   MSE = 0.000885

Input    Target   Output   Predicted
[0 0]      0     0.0227      0
[0 1]      1     0.9690      1
[1 0]      1     0.9746      1
[1 1]      0     0.0376      0
Accuracy = 100.00%
>>>
```

9. Result / Conclusion

The Backpropagation algorithm was implemented successfully. The mean squared error decreased from 0.019769 at the first logged epoch to 0.000885 at epoch 5000 and the network achieved accuracy = 100.00% on its evaluation data.

Subproblem 2: Half Adder (Multiple Outputs)

1. Aim

To train a 2-4-2 network with Backpropagation to behave as a half adder that produces the Sum and Carry bits.

2. Concept

The Sum output is the XOR of the two inputs and the Carry output is their AND. Having two output neurons shows how output error terms are computed for every neuron in the output layer.

3. Dataset Description

The training examples are the four input combinations of a half adder (Sum = A XOR B, Carry = A AND B).

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

4. Algorithm

1. Initialise the weights randomly between −1 and 1, set all biases to 0, and choose the learning rate and number of epochs.
2. Present one training sample and perform the forward pass: compute the hidden-layer outputs and then the output-layer outputs using the sigmoid function.
3. Compute the error at each output neuron and convert it to the output error term δ.
4. Propagate δ backwards through the weights to obtain the hidden-layer error terms.
5. Update every weight and bias by gradient descent: new value = old value + η × δ × input.
6. Repeat steps 2–5 for every training sample; one pass over the data is one epoch. Continue until the chosen number of epochs is complete.
7. Test the trained network on the test inputs and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Network: 2 input neurons, 4 hidden neurons, 2 output neuron(s); sigmoid activation; learning rate η = 0.5; weights start random in [−1, 1] (seed 1) and biases start at 0.
2. Take training sample 4: x = (1.0000, 1.0000), target t = (0.0000, 1.0000).
3. Hidden neuron h1: z = (−0.1660)(1.0000) + (−0.7065)(1.0000) + 0.0000 = −0.8724; a = σ(z) = 0.2947.
4. Hidden neuron h2: z = (0.4406)(1.0000) + (−0.8153)(1.0000) + 0.0000 = −0.3747; a = σ(z) = 0.4074.
5. Hidden neuron h3: z = (−0.9998)(1.0000) + (−0.6275)(1.0000) + 0.0000 = −1.6273; a = σ(z) = 0.1642.

6. The remaining 1 hidden neuron(s) are computed in the same way; the hidden vector is a = (0.2947, 0.4074, 0.1642, 0.3309).
7. Output neuron o1: z = Σ(a_h × w) + b = −0.5365; a = σ(z) = 0.3690.
8. Output neuron o2: z = Σ(a_h × w) + b = 0.4108; a = σ(z) = 0.6013.
9. Output error term δo1 = (t − a)·a·(1 − a) = (0.0000 − 0.3690)×0.3690×(1 − 0.3690) = −0.0859.
10. Output error term δo2 = (t − a)·a·(1 − a) = (1.0000 − 0.6013)×0.6013×(1 − 0.6013) = 0.0956.
11. Hidden error term δh1 = a(1 − a)·Σ(δo × w) = 0.2947×(1 − 0.2947)×(−0.0859×−0.2065 + …) = 0.0052.
12. Hidden error term δh2 = a(1 − a)·Σ(δo × w) = 0.4074×(1 − 0.4074)×(−0.0859×−0.1616 + …) = 0.0119.
13. Hidden error term δh3 = a(1 − a)·Σ(δo × w) = 0.1642×(1 − 0.1642)×(−0.0859×−0.5911 + …) = 0.0169.
14. Weight update (output layer): w(h1→o1) new = −0.2065 + 0.5×−0.0859×0.2947 = −0.2191.
15. Weight update (hidden layer): w(x1→h1) new = −0.1660 + 0.5×0.0052×1.0000 = −0.1633. Biases are updated as b new = b + η×δ.
16. These forward and backward passes are repeated for every training sample in every epoch. The mean squared error falls from 0.004401 (epoch 1000) to 0.000424 (epoch 5000).
17. After training, the test inputs are passed forward only (no weight changes). Output > 0.5 (or the largest output neuron) gives the predicted class: accuracy = 100.00%.

6. Formulae

Notation: x_i = input, a_j = neuron output, t_k = target, η = learning rate, w_{ij} = weight from neuron i to neuron j.

- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$, with derivative $\sigma'(z) = a(1 - a)$
- Forward pass: $z_j = \Sigma\, w_{ij}\, x_i + b_j$; $a_j = \sigma(z_j)$
- Output error term: $\delta_k = (t_k - a_k)\, a_k\, (1 - a_k)$
- Hidden error term: $\delta_j = a_j\, (1 - a_j)\, \Sigma_k\, \delta_k\, w_{jk}$
- Weight update: $w_{ij}(\text{new}) = w_{ij} + \eta\, \delta_j\, x_i$; $b_j(\text{new}) = b_j + \eta\, \delta_j$
- Mean squared error: $MSE = (1/N)\, \Sigma\, (t - a)^2$

7. Python Program

```
import numpy as np

np.random.seed(1)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

class NeuralNetwork:
```

```
    def __init__(self, n_in, n_hidden, n_out, lr):
        self.W1 = np.random.uniform(-1, 1, (n_in, n_hidden))
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = np.random.uniform(-1, 1, (n_hidden, n_out))
        self.b2 = np.zeros((1, n_out))
        self.lr = lr

    def forward(self, X):
        self.a1 = sigmoid(X @ self.W1 + self.b1)       # hidden layer
        self.a2 = sigmoid(self.a1 @ self.W2 + self.b2)  # output layer
        return self.a2

    def backward(self, x, y):
        delta2 = (y - self.a2) * self.a2 * (1 - self.a2)          # output
error term
        delta1 = (delta2 @ self.W2.T) * self.a1 * (1 - self.a1)   # hidden
error term
        self.W2 += self.lr * self.a1.T @ delta2
        self.b2 += self.lr * delta2
        self.W1 += self.lr * x.T @ delta1
        self.b1 += self.lr * delta1

    def train(self, X, y, epochs, log_every):
        for epoch in range(1, epochs + 1):
            for i in np.random.permutation(len(X)):
                xi, yi = X[i:i + 1], y[i:i + 1]
                self.forward(xi)
                self.backward(xi, yi)
            if epoch % log_every == 0:
                mse = np.mean((y - self.forward(X)) ** 2)
                print("Epoch %5d   MSE = %.6f" % (epoch, mse))

# Half adder: inputs (A, B) -> outputs (Sum, Carry)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0, 0], [1, 0], [1, 0], [0, 1]], dtype=float)

nn = NeuralNetwork(n_in=2, n_hidden=4, n_out=2, lr=0.5)
nn.train(X, y, epochs=5000, log_every=1000)

out = nn.forward(X)
print("\nA B   Target(S,C)   Output(S,C)         Predicted(S,C)")
for xi, t, o in zip(X, y, out):
    print(xi.astype(int)[0], xi.astype(int)[1], "    ", t.astype(int), "
",
          np.round(o, 4), "  ", (o > 0.5).astype(int))
print("Accuracy = %.2f%%" % (np.mean((out > 0.5) == y) * 100))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/4.2.py ====
Epoch  1000   MSE = 0.004401
Epoch  2000   MSE = 0.001384
Epoch  3000   MSE = 0.000798
Epoch  4000   MSE = 0.000556
Epoch  5000   MSE = 0.000424

A B   Target(S,C)   Output(S,C)          Predicted(S,C)
0 0      [0 0]       [0.019  0.0001]     [0 0]
0 1      [1 0]       [0.9756 0.0201]     [1 0]
1 0      [1 0]       [0.9808 0.01  ]     [1 0]
1 1      [0 1]       [0.0291 0.9732]     [0 1]
Accuracy = 100.00%
>>>
```

9. Result / Conclusion

The Backpropagation algorithm was implemented successfully. The mean squared error decreased from 0.004401 at the first logged epoch to 0.000424 at epoch 5000 and the network achieved accuracy = 100.00% on its evaluation data.

Subproblem 3: Student Pass/Fail Prediction

1. Aim

To train a 2-4-1 network with Backpropagation to predict whether a student passes from study hours and attendance, and to test it on unseen students.

2. Concept

The inputs are study hours per day and attendance percentage. Both are scaled to the range 0–1 so that neither dominates the weight updates. The target is 1 for Pass and 0 for Fail.

3. Dataset Description

Training data (12 students):

Study hours/day	Attendance (%)	Result
1	45	Fail
2	60	Fail
3	50	Fail
3	80	Fail
4	65	Fail
5	60	Fail
5	75	Pass
6	85	Pass
7	70	Pass
8	90	Pass
9	95	Pass
4	90	Pass

Test data (4 unseen students):

Study hours/day	Attendance (%)	Result
2	70	Fail
6	90	Pass
4	55	Fail
7	95	Pass

4. Algorithm

1. Initialise the weights randomly between −1 and 1, set all biases to 0, and choose the learning rate and number of epochs.
2. Present one training sample and perform the forward pass: compute the hidden-layer outputs and then the output-layer outputs using the sigmoid function.
3. Compute the error at each output neuron and convert it to the output error term δ.
4. Propagate δ backwards through the weights to obtain the hidden-layer error terms.
5. Update every weight and bias by gradient descent: new value = old value + η × δ × input.

6. Repeat steps 2–5 for every training sample; one pass over the data is one epoch. Continue until the chosen number of epochs is complete.
7. Test the trained network on the test inputs and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Network: 2 input neurons, 4 hidden neurons, 1 output neuron(s); sigmoid activation; learning rate $\eta = 0.5$; weights start random in $[-1, 1]$ (seed 1) and biases start at 0.
2. Scaling: study hours ÷ 10 and attendance ÷ 100, so the first student (1 hour, 45%) becomes $x = (0.1, 0.45)$.
3. Take training sample 1: $x = (0.1000, 0.4500)$, target $t = (0.0000)$.
4. Hidden neuron h1: $z = (-0.1660)(0.1000) + (-0.7065)(0.4500) + 0.0000 = -0.3345$; $a = \sigma(z) = 0.4171$.
5. Hidden neuron h2: $z = (0.4406)(0.1000) + (-0.8153)(0.4500) + 0.0000 = -0.3228$; $a = \sigma(z) = 0.4200$.
6. Hidden neuron h3: $z = (-0.9998)(0.1000) + (-0.6275)(0.4500) + 0.0000 = -0.3823$; $a = \sigma(z) = 0.4056$.
7. The remaining 1 hidden neuron(s) are computed in the same way; the hidden vector is $a = (0.4171, 0.4200, 0.4056, 0.4555)$.
8. Output neuron o1: $z = \Sigma(a_h \times w) + b = 0.0497$; $a = \sigma(z) = 0.5124$.
9. Output error term $\delta o1 = (t - a) \cdot a \cdot (1 - a) = (0.0000 - 0.5124) \times 0.5124 \times (1 - 0.5124) = -0.1280$.
10. Hidden error term $\delta h1 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.4171 \times (1 - 0.4171) \times (-0.1280 \times -0.2065) = 0.0064$.
11. Hidden error term $\delta h2 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.4200 \times (1 - 0.4200) \times (-0.1280 \times 0.0776) = -0.0024$.
12. Hidden error term $\delta h3 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.4056 \times (1 - 0.4056) \times (-0.1280 \times -0.1616) = 0.0050$.
13. Weight update (output layer): $w(h1 \rightarrow o1)$ new $= -0.2065 + 0.5 \times -0.1280 \times 0.4171 = -0.2332$.
14. Weight update (hidden layer): $w(x1 \rightarrow h1)$ new $= -0.1660 + 0.5 \times 0.0064 \times 0.1000 = -0.1656$. Biases are updated as b new $= b + \eta \times \delta$.
15. These forward and backward passes are repeated for every training sample in every epoch. The mean squared error falls from 0.012696 (epoch 500) to 0.001414 (epoch 2000).
16. After training, the test inputs are passed forward only (no weight changes). Output > 0.5 (or the largest output neuron) gives the predicted class: accuracy = 100.00%.

6. Formulae

Notation: x_i = input, a_j = neuron output, t_k = target, η = learning rate, w_{ij} = weight from neuron i to neuron j.

- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$, with derivative $\sigma'(z) = a(1 - a)$

- Forward pass: $z_j = \Sigma\, w_{ij}\, x_i + b_j$; $a_j = \sigma(z_j)$
- Output error term: $\delta_k = (t_k - a_k)\, a_k\, (1 - a_k)$
- Hidden error term: $\delta_j = a_j\, (1 - a_j)\, \Sigma_k\, \delta_k\, w_{jk}$
- Weight update: $w_{ij}(new) = w_{ij} + \eta\, \delta_j\, x_i$; $b_j(new) = b_j + \eta\, \delta_j$
- Mean squared error: $MSE = (1/N)\, \Sigma\, (t - a)^2$

7. Python Program

```
import numpy as np

np.random.seed(1)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

class NeuralNetwork:
    def __init__(self, n_in, n_hidden, n_out, lr):
        self.W1 = np.random.uniform(-1, 1, (n_in, n_hidden))
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = np.random.uniform(-1, 1, (n_hidden, n_out))
        self.b2 = np.zeros((1, n_out))
        self.lr = lr

    def forward(self, X):
        self.a1 = sigmoid(X @ self.W1 + self.b1)       # hidden layer
        self.a2 = sigmoid(self.a1 @ self.W2 + self.b2)  # output layer
        return self.a2

    def backward(self, x, y):
        delta2 = (y - self.a2) * self.a2 * (1 - self.a2)          # output
error term
        delta1 = (delta2 @ self.W2.T) * self.a1 * (1 - self.a1)   # hidden
error term
        self.W2 += self.lr * self.a1.T @ delta2
        self.b2 += self.lr * delta2
        self.W1 += self.lr * x.T @ delta1
        self.b1 += self.lr * delta1

    def train(self, X, y, epochs, log_every):
        for epoch in range(1, epochs + 1):
            for i in np.random.permutation(len(X)):
                xi, yi = X[i:i + 1], y[i:i + 1]
                self.forward(xi)
                self.backward(xi, yi)
            if epoch % log_every == 0:
                mse = np.mean((y - self.forward(X)) ** 2)
                print("Epoch %5d   MSE = %.6f" % (epoch, mse))

# Student pass/fail: [study hours, attendance %] -> 1 = Pass, 0 = Fail
X_train = np.array([[1, 45], [2, 60], [3, 50], [3, 80], [4, 65], [5, 60],
                    [5, 75], [6, 85], [7, 70], [8, 90], [9, 95], [4, 90]],
dtype=float)
y_train = np.array([[0], [0], [0], [0], [0], [0], [1], [1], [1], [1], [1],
[1]], dtype=float)
X_test = np.array([[2, 70], [6, 90], [4, 55], [7, 95]], dtype=float)
y_test = np.array([[0], [1], [0], [1]], dtype=float)

scale = np.array([10.0, 100.0])          # normalise both features to the
range 0-1
```

```
X_train, X_test = X_train / scale, X_test / scale

nn = NeuralNetwork(n_in=2, n_hidden=4, n_out=1, lr=0.5)
nn.train(X_train, y_train, epochs=2000, log_every=500)

out = nn.forward(X_test)
print("\nHours  Attendance  Actual  Output   Predicted")
for xi, t, o in zip(X_test * scale, y_test, out):
    print("%3d    %6d%%      %d     %.4f      %d" % (xi[0], xi[1], t[0],
o[0], o[0] > 0.5))
print("Test accuracy = %.2f%%" % (np.mean((out > 0.5) == y_test) * 100))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/4.3.py ====
Epoch   500   MSE = 0.012696
Epoch  1000   MSE = 0.004070
Epoch  1500   MSE = 0.002167
Epoch  2000   MSE = 0.001414

Hours  Attendance  Actual  Output   Predicted
  2        70%       0     0.0002      0
  6        90%       1     0.9993      1
  4        55%       0     0.0008      0
  7        95%       1     0.9998      1
Test accuracy = 100.00%
>>>
```

9. Result / Conclusion

The Backpropagation algorithm was implemented successfully. The mean squared error decreased from 0.012696 at the first logged epoch to 0.001414 at epoch 2000 and the network achieved accuracy = 100.00% on its evaluation data.

Subproblem 4: Iris Flower Classification (Multi-class)

1. Aim

To train a 4-6-3 network with Backpropagation to classify Iris flowers into three species and measure the accuracy on a held-out test set.

2. Concept

The four measurements are scaled to 0–1 and the species is one-hot encoded, so each output neuron represents one species. The predicted class is the neuron with the largest output.

3. Dataset Description

The Iris dataset (built into scikit-learn) has 150 samples, 4 numeric features, and 3 classes with 50 samples each. Six representative rows are shown below.

Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.3	3.3	6	2.5	virginica
5.8	2.7	5.1	1.9	virginica

4. Algorithm

1. Initialise the weights randomly between −1 and 1, set all biases to 0, and choose the learning rate and number of epochs.
2. Present one training sample and perform the forward pass: compute the hidden-layer outputs and then the output-layer outputs using the sigmoid function.
3. Compute the error at each output neuron and convert it to the output error term δ.
4. Propagate δ backwards through the weights to obtain the hidden-layer error terms.
5. Update every weight and bias by gradient descent: new value = old value + η × δ × input.
6. Repeat steps 2–5 for every training sample; one pass over the data is one epoch. Continue until the chosen number of epochs is complete.
7. Test the trained network on the test inputs and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Network: 4 input neurons, 6 hidden neurons, 3 output neuron(s); sigmoid activation; learning rate η = 0.2; weights start random in [−1, 1] (seed 1) and biases start at 0.
2. Scaling: each feature is min–max scaled, (x − min)/(max − min). Species are one-hot encoded: setosa = (1,0,0), versicolor = (0,1,0), virginica = (0,0,1). The data is split 120 training / 30 test.
3. Take training sample 1: x = (0.4167, 0.8333, 0.0339, 0.0417), target t = (1.0000, 0.0000, 0.0000).
4. Hidden neuron h1: $z = \Sigma(w_i x_i) + b$ over 4 inputs = −0.4944; $a = \sigma(z) = 0.3788$.

5. Hidden neuron h2: $z = \Sigma(w_i x_i) + b$ over 4 inputs = 0.3619; $a = \sigma(z) = 0.5895$.
6. Hidden neuron h3: $z = \Sigma(w_i x_i) + b$ over 4 inputs = 0.6637; $a = \sigma(z) = 0.6601$.
7. The remaining 3 hidden neuron(s) are computed in the same way; the hidden vector is a = (0.3788, 0.5895, 0.6601, 0.3561, 0.6977, 0.5217).
8. Output neuron o1: $z = \Sigma(a_h \times w) + b = -1.1715$; $a = \sigma(z) = 0.2366$.
9. Output neuron o2: $z = \Sigma(a_h \times w) + b = -0.9448$; $a = \sigma(z) = 0.2799$.
10. Output neuron o3: $z = \Sigma(a_h \times w) + b = -0.2123$; $a = \sigma(z) = 0.4471$.
11. Output error term $\delta o1 = (t - a) \cdot a \cdot (1 - a) = (1.0000 - 0.2366) \times 0.2366 \times (1 - 0.2366) = 0.1379$.
12. Output error term $\delta o2 = (t - a) \cdot a \cdot (1 - a) = (0.0000 - 0.2799) \times 0.2799 \times (1 - 0.2799) = -0.0564$.
13. Output error term $\delta o3 = (t - a) \cdot a \cdot (1 - a) = (0.0000 - 0.4471) \times 0.4471 \times (1 - 0.4471) = -0.1105$.
14. Hidden error term $\delta h1 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.3788 \times (1 - 0.3788) \times (0.1379 \times -0.4416 + \ldots) = -0.0410$.
15. Hidden error term $\delta h2 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.5895 \times (1 - 0.5895) \times (0.1379 \times 0.1221 + \ldots) = 0.0011$.
16. Hidden error term $\delta h3 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.6601 \times (1 - 0.6601) \times (0.1379 \times -0.5341 + \ldots) = -0.0187$.
17. Weight update (output layer): $w(h1 \rightarrow o1)$ new = $-0.4416 + 0.2 \times 0.1379 \times 0.3788 = -0.4312$.
18. Weight update (hidden layer): $w(x1 \rightarrow h1)$ new = $0.2467 + 0.2 \times -0.0410 \times 0.4167 = 0.2433$. Biases are updated as b new = $b + \eta \times \delta$.
19. These forward and backward passes are repeated for every training sample in every epoch. The mean squared error falls from 0.008945 (epoch 200) to 0.004744 (epoch 600).
20. After training, the test inputs are passed forward only (no weight changes). Output > 0.5 (or the largest output neuron) gives the predicted class: accuracy = 90.00%.

6. Formulae

Notation: x_i = input, a_j = neuron output, t_k = target, η = learning rate, w_{ij} = weight from neuron i to neuron j.

- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$, with derivative $\sigma'(z) = a(1 - a)$
- Forward pass: $z_j = \Sigma\, w_{ij}\, x_i + b_j$; $a_j = \sigma(z_j)$
- Output error term: $\delta_k = (t_k - a_k)\, a_k\, (1 - a_k)$
- Hidden error term: $\delta_j = a_j\, (1 - a_j)\, \Sigma_k\, \delta_k\, w_{jk}$
- Weight update: $w_{ij}(new) = w_{ij} + \eta\, \delta_j\, x_i$; $b_j(new) = b_j + \eta\, \delta_j$
- Mean squared error: $MSE = (1/N)\, \Sigma\, (t - a)^2$

7. Python Program

```
import numpy as np
```

```
np.random.seed(1)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

class NeuralNetwork:
    def __init__(self, n_in, n_hidden, n_out, lr):
        self.W1 = np.random.uniform(-1, 1, (n_in, n_hidden))
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = np.random.uniform(-1, 1, (n_hidden, n_out))
        self.b2 = np.zeros((1, n_out))
        self.lr = lr

    def forward(self, X):
        self.a1 = sigmoid(X @ self.W1 + self.b1)       # hidden layer
        self.a2 = sigmoid(self.a1 @ self.W2 + self.b2)  # output layer
        return self.a2

    def backward(self, x, y):
        delta2 = (y - self.a2) * self.a2 * (1 - self.a2)          # output
error term
        delta1 = (delta2 @ self.W2.T) * self.a1 * (1 - self.a1)   # hidden
error term
        self.W2 += self.lr * self.a1.T @ delta2
        self.b2 += self.lr * delta2
        self.W1 += self.lr * x.T @ delta1
        self.b1 += self.lr * delta1

    def train(self, X, y, epochs, log_every):
        for epoch in range(1, epochs + 1):
            for i in np.random.permutation(len(X)):
                xi, yi = X[i:i + 1], y[i:i + 1]
                self.forward(xi)
                self.backward(xi, yi)
            if epoch % log_every == 0:
                mse = np.mean((y - self.forward(X)) ** 2)
                print("Epoch %5d   MSE = %.6f" % (epoch, mse))

from sklearn.datasets import load_iris

iris = load_iris()
X, labels = iris.data, iris.target
X = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))   # min-max scaling
y = np.eye(3)[labels]                                       # one-hot targets

idx = np.random.permutation(len(X))
train_idx, test_idx = idx[:120], idx[120:]                  # 120 train / 30
test

nn = NeuralNetwork(n_in=4, n_hidden=6, n_out=3, lr=0.2)
nn.train(X[train_idx], y[train_idx], epochs=600, log_every=200)

pred = np.argmax(nn.forward(X[test_idx]), axis=1)
actual = labels[test_idx]
print("\nFirst 5 test samples")
for p, a in list(zip(pred, actual))[:5]:
    print("Actual:", iris.target_names[a], "  Predicted:",
iris.target_names[p])
print("Test accuracy = %.2f%%" % (np.mean(pred == actual) * 100))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/4.4.py ====
Epoch   200   MSE = 0.008945
Epoch   400   MSE = 0.006167
Epoch   600   MSE = 0.004744

First 5 test samples
Actual: setosa   Predicted: setosa
Actual: virginica   Predicted: virginica
Actual: setosa   Predicted: setosa
Actual: versicolor   Predicted: versicolor
Actual: setosa   Predicted: setosa
Test accuracy = 90.00%
>>>
```

9. Result / Conclusion

The Backpropagation algorithm was implemented successfully. The mean squared error decreased from 0.008945 at the first logged epoch to 0.004744 at epoch 600 and the network achieved accuracy = 90.00% on its evaluation data.

Subproblem 5: Breast Cancer Wisconsin Dataset (Binary)

1. Aim

To train a 30-8-1 network with Backpropagation on the Breast Cancer Wisconsin dataset and test its accuracy on unseen records.

2. Concept

Each record has 30 numeric measurements. They are standardised using the training mean and standard deviation. The single output neuron gives a value near 1 for the benign class and near 0 for the malignant class.

3. Dataset Description

The Breast Cancer Wisconsin dataset (built into scikit-learn) has 569 records, 30 numeric features, and 2 classes (212 malignant, 357 benign). The first four of the 30 features are shown for five records.

Mean radius	Mean texture	Mean perimeter	Mean area	Class
17.99	10.38	122.8	1001	Malignant
20.57	17.77	132.9	1326	Malignant
19.69	21.25	130	1203	Malignant
11.42	20.38	77.58	386.1	Malignant
20.29	14.34	135.1	1297	Malignant

4. Algorithm

1. Initialise the weights randomly between −1 and 1, set all biases to 0, and choose the learning rate and number of epochs.
2. Present one training sample and perform the forward pass: compute the hidden-layer outputs and then the output-layer outputs using the sigmoid function.
3. Compute the error at each output neuron and convert it to the output error term δ.
4. Propagate δ backwards through the weights to obtain the hidden-layer error terms.
5. Update every weight and bias by gradient descent: new value = old value + η × δ × input.
6. Repeat steps 2–5 for every training sample; one pass over the data is one epoch. Continue until the chosen number of epochs is complete.
7. Test the trained network on the test inputs and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Network: 30 input neurons, 8 hidden neurons, 1 output neuron(s); sigmoid activation; learning rate η = 0.05; weights start random in [−1, 1] (seed 1) and biases start at 0.
2. Scaling: each of the 30 features is standardised, (x − mean)/std, using training data only. The data is split 455 training / 114 test.
3. Take training sample 1: x = (0.1598, −1.2675, 0.2559, 0.0028, …), target t = (1.0000).
4. Hidden neuron h1: $z = \Sigma(w_i x_i) + b$ over 30 inputs = 2.8791; $a = \sigma(z) = 0.9468$.

5. Hidden neuron h2: $z = \Sigma(w_i x_i) + b$ over 30 inputs = −0.5724; $a = \sigma(z) = 0.3607$.
6. Hidden neuron h3: $z = \Sigma(w_i x_i) + b$ over 30 inputs = 5.6363; $a = \sigma(z) = 0.9964$.
7. The remaining 5 hidden neuron(s) are computed in the same way; the hidden vector is a = (0.9468, 0.3607, 0.9964, 0.0050, 0.8825, 0.3318, 0.1249, 0.3187).
8. Output neuron o1: $z = \Sigma(a_h \times w) + b = -1.3596$; $a = \sigma(z) = 0.2043$.
9. Output error term $\delta o1 = (t - a) \cdot a \cdot (1 - a) = (1.0000 - 0.2043) \times 0.2043 \times (1 - 0.2043) = 0.1293$.
10. Hidden error term $\delta h1 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.9468 \times (1 - 0.9468) \times (0.1293 \times -0.8398) = -0.0055$.
11. Hidden error term $\delta h2 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.3607 \times (1 - 0.3607) \times (0.1293 \times 0.8093) = 0.0241$.
12. Hidden error term $\delta h3 = a(1 - a) \cdot \Sigma(\delta o \times w) = 0.9964 \times (1 - 0.9964) \times (0.1293 \times -0.2597) = -0.0001$.
13. Weight update (output layer): $w(h1 \rightarrow o1)$ new = $-0.8398 + 0.05 \times 0.1293 \times 0.9468 = -0.8337$.
14. Weight update (hidden layer): $w(x1 \rightarrow h1)$ new = $0.9189 + 0.05 \times -0.0055 \times 0.1598 = 0.9188$. Biases are updated as b new = $b + \eta \times \delta$.
15. These forward and backward passes are repeated for every training sample in every epoch. The mean squared error falls from 0.027137 (epoch 10) to 0.011957 (epoch 50).
16. After training, the test inputs are passed forward only (no weight changes). Output > 0.5 (or the largest output neuron) gives the predicted class: accuracy = 97.37%.

6. Formulae

Notation: x_i = input, a_j = neuron output, t_k = target, η = learning rate, w_{ij} = weight from neuron i to neuron j.

- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$, with derivative $\sigma'(z) = a(1 - a)$
- Forward pass: $z_j = \Sigma\, w_{ij}\, x_i + b_j$; $a_j = \sigma(z_j)$
- Output error term: $\delta_k = (t_k - a_k)\, a_k\, (1 - a_k)$
- Hidden error term: $\delta_j = a_j\, (1 - a_j)\, \Sigma_k\, \delta_k\, w_{jk}$
- Weight update: $w_{ij}(new) = w_{ij} + \eta\, \delta_j\, x_i$; $b_j(new) = b_j + \eta\, \delta_j$
- Mean squared error: $MSE = (1/N)\, \Sigma\, (t - a)^2$

7. Python Program

```
import numpy as np

np.random.seed(1)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

class NeuralNetwork:
    def __init__(self, n_in, n_hidden, n_out, lr):
        self.W1 = np.random.uniform(-1, 1, (n_in, n_hidden))
        self.b1 = np.zeros((1, n_hidden))
```

```
        self.W2 = np.random.uniform(-1, 1, (n_hidden, n_out))
        self.b2 = np.zeros((1, n_out))
        self.lr = lr

    def forward(self, X):
        self.a1 = sigmoid(X @ self.W1 + self.b1)       # hidden layer
        self.a2 = sigmoid(self.a1 @ self.W2 + self.b2)  # output layer
        return self.a2

    def backward(self, x, y):
        delta2 = (y - self.a2) * self.a2 * (1 - self.a2)          # output
error term
        delta1 = (delta2 @ self.W2.T) * self.a1 * (1 - self.a1)   # hidden
error term
        self.W2 += self.lr * self.a1.T @ delta2
        self.b2 += self.lr * delta2
        self.W1 += self.lr * x.T @ delta1
        self.b1 += self.lr * delta1

    def train(self, X, y, epochs, log_every):
        for epoch in range(1, epochs + 1):
            for i in np.random.permutation(len(X)):
                xi, yi = X[i:i + 1], y[i:i + 1]
                self.forward(xi)
                self.backward(xi, yi)
            if epoch % log_every == 0:
                mse = np.mean((y - self.forward(X)) ** 2)
                print("Epoch %5d   MSE = %.6f" % (epoch, mse))

from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer()
X, y = cancer.data, cancer.target.reshape(-1, 1).astype(float)

idx = np.random.permutation(len(X))
train_idx, test_idx = idx[:455], idx[455:]                  # 455 train / 114
test
mean, std = X[train_idx].mean(axis=0), X[train_idx].std(axis=0)
X = (X - mean) / std                                        # standardise
using training data

nn = NeuralNetwork(n_in=30, n_hidden=8, n_out=1, lr=0.05)
nn.train(X[train_idx], y[train_idx], epochs=50, log_every=10)

out = nn.forward(X[test_idx])
pred = (out > 0.5).astype(int)
print("\nTest samples        :", len(test_idx))
print("Correct predictions :", int(np.sum(pred == y[test_idx])))
print("Test accuracy = %.2f%%" % (np.mean(pred == y[test_idx]) * 100))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/4.5.py ====
Epoch    10   MSE = 0.027137
Epoch    20   MSE = 0.018704
Epoch    30   MSE = 0.015126
Epoch    40   MSE = 0.013155
Epoch    50   MSE = 0.011957

Test samples        : 114
Correct predictions : 111
Test accuracy = 97.37%
>>>
```

9. Result / Conclusion

The Backpropagation algorithm was implemented successfully. The mean squared error decreased from 0.027137 at the first logged epoch to 0.011957 at epoch 50 and the network achieved accuracy = 97.37% on its evaluation data.

Overall Result of Experiment 4

A neural network trained with Backpropagation was tested on five datasets. The final accuracy on the evaluation data of each program is shown below.

Subproblem	Application	Network (in-hidden-out)	Accuracy
1	XOR Gate Learning	2-4-1	100.00%
2	Half Adder (Multiple Outputs)	2-4-2	100.00%
3	Student Pass/Fail Prediction	2-4-1	100.00%
4	Iris Flower Classification (Multi-class)	4-6-3	90.00%
5	Breast Cancer Wisconsin Dataset (Binary)	30-8-1	97.37%

Final conclusion: the Backpropagation algorithm lets a network with one hidden layer learn both non-linearly separable problems such as XOR and real datasets such as Iris and Breast Cancer. The error decreased steadily during training in every subproblem. Accuracy on small test sets is only an estimate and may change with a different random split or seed.

Notation: the outputs shown are the actual outputs of the programs listed in this record for the datasets given here.